

○ PYTHON FUNDAMENTOS · AULA 1 · GUIA DE APRENDIZAGEM

GUIA COMPLETO

Python

Do zero ao avançado: fundamentos, estruturas, boas práticas e aplicações

27 capítulos

Fundamentos

Intermediário

Avançado

Aplicações práticas

Atualizado em Março 2026 | Python 3.13/3.14

Por Rodrigo | Gerado com Claude

Sumário

PARTE 1 · FUNDAMENTOS

- 1 O Que é Python e Por Que Aprender
- 2 Instalação, Ambiente e Ferramentas
- 3 Variáveis, Tipos de Dados e Operadores
- 4 Strings: Manipulação e Formatação
- 5 Estruturas Condicionais (if/elif/else)
- 6 Estruturas de Repetição (for/while)
- 7 Funções e Escopo
- 8 Listas, Tuplas e Conjuntos
- 9 Dicionários
- 10 Comprehensions (List, Dict, Set)

PARTE 2 · INTERMEDIÁRIO

- 11 Manipulação de Arquivos
- 12 Tratamento de Erros (try/except)
- 13 Módulos e Pacotes (import, pip)
- 14 Programação Orientada a Objetos (POO)
- 15 Decoradores e Geradores
- 16 Expressões Lambda e Funções de Alta Ordem
- 17 Type Hints e Tipagem Estática

PARTE 3 · AVANÇADO

- 18 Ambientes Virtuais e Gerenciamento de Dependências
- 19 Programação Assíncrona (async/await)
- 20 Testes Automatizados (pytest)
- 21 Novidades do Python 3.13 e 3.14

PARTE 4 · APLICAÇÕES PRÁTICAS

- 22 Ciência de Dados (Pandas, NumPy, Matplotlib)
- 23 Desenvolvimento Web (FastAPI, Django, Flask)
- 24 Automação e Web Scraping
- 25 IA e Machine Learning (scikit-learn, LangChain)
- 26 APIs REST com FastAPI
- 27 Boas Práticas e Produtividade

01 O Que é Python e Por Que Aprender

Python é uma linguagem de programação de alto nível, interpretada, dinamicamente tipada e com sintaxe limpa e legível. Criada por Guido van Rossum em 1991, Python se tornou a linguagem mais popular do mundo em 2026, sendo usada em ciência de dados, IA, web, automação, DevOps e muito mais.

MOTIVO	DETALHES
Simplicidade	Sintaxe clara, fácil de ler e escrever. Ideal para iniciantes e produtividade
Versatilidade	Web, dados, IA, automação, games, IoT, desktop - Python faz tudo
Ecossistema	400.000+ pacotes no PyPI. Biblioteca para qualquer necessidade
Mercado	Linguagem mais demandada em vagas de tecnologia globalmente
Comunidade	Milhões de desenvolvedores, documentação excelente, muito conteúdo
IA e Dados	Linguagem padrão para Machine Learning, Data Science e LLMs

Python no ranking de linguagens (2026):

RANKING	LINGUAGEM	USO PRINCIPAL
1º	Python	IA, dados, web, automação, scripts
2º	JavaScript	Frontend web, Node.js
3º	TypeScript	Frontend/backend tipado
4º	Java	Enterprise, Android
5º	C/C++	Sistemas, embedded, games

Dica: Python é a melhor primeira linguagem para quem está começando. Também é a mais usada por profissionais que não são programadores (analistas, cientistas, marketeiros).

02 Instalação, Ambiente e Ferramentas

Configurar o ambiente corretamente é o primeiro passo para uma boa experiência com Python.

Instalação:

```
# Windows: Baixe de python.org (marque 'Add to PATH')
# macOS:
brew install python

# Linux (Ubuntu/Debian):
sudo apt update && sudo apt install python3 python3-pip

# Verificar instalação:
python --version  # Python 3.14.x
pip --version     # pip 24.x
```

FERRAMENTA	TIPO	POR QUE USAR
VS Code	Editor/IDE	Leve, extensões Python excelentes, debug integrado
PyCharm	IDE	IDE completa, refatoração avançada, debug profissional
Jupyter Notebook	Notebook	Análise de dados, experimentação, visualização
uv	Gerenciador de pacotes	Substituto moderno do pip (10-100x mais rápido)
Ruff	Linter + Formatter	Substitui flake8/black/isort. Extremamente rápido
pytest	Testes	Framework de testes padrão do ecossistema Python
mypy	Type checker	Verificação estática de tipos

Dica: Em 2026, use 'uv' em vez de 'pip' para instalar pacotes. É até 100x mais rápido e gerencia ambientes virtuais automaticamente: `uv init`, `uv add pandas`.

03 Variáveis, Tipos de Dados e Operadores

Variáveis em Python são criadas por atribuição simples. Não é necessário declarar o tipo - Python é dinamicamente tipado.

```
# Variáveis e tipos básicos
nome = 'Rodrigo' # str (texto)
idade = 30       # int (inteiro)
altura = 1.75    # float (decimal)
ativo = True     # bool (booleano)
nada = None      # NoneType (nulo)

# Verificar tipo
print(type(nome)) # <class 'str'>
print(type(idade)) # <class 'int'>
```

Tipos de dados:

TIPO	EXEMPLO	MUTÁVEL	DESCRIÇÃO
int	42, -7, 0	Não	Números inteiros (sem limite de tamanho)
float	3.14, -0.5	Não	Números decimais (ponto flutuante)
str	'Olá', "mundo"	Não	Texto (sequência de caracteres)
bool	True, False	Não	Booleano (verdadeiro/falso)
list	[1, 2, 3]	Sim	Lista ordenada (pode alterar)
tuple	(1, 2, 3)	Não	Tupla ordenada (imutável)
dict	{'a': 1, 'b': 2}	Sim	Dicionário (chave: valor)
set	{1, 2, 3}	Sim	Conjunto (sem duplicatas, sem ordem)
NoneType	None	N/A	Ausência de valor

Operadores:

TIPO	OPERADORES	EXEMPLO
Aritméticos	+ - * / // % **	10 / 3 = 3.33 · 10 // 3 = 3 · 2**8 = 256
Comparação	== != > < >= <=	5 > 3 → True
Lógicos	and or not	True and False → False
Atribuição	= += -= *= /=	x += 5 (mesmo que x = x + 5)
Identidade	is is not	x is None → True
Pertinência	in not in	'a' in 'abc' → True

Dica: Use // para divisão inteira e % para resto. Use ** para potenciação (2**10 = 1024). Evite 'is' para comparar valores; use '==' (is compara identidade, não valor).

04 Strings: Manipulação e Formatação

Strings são um dos tipos mais usados em Python. Dominar manipulação de strings é essencial para qualquer aplicação.

```
# Criação de strings
s1 = 'aspas simples'
s2 = "aspas duplas"
s3 = '''string
multilinhas'''

# f-strings (formatação moderna - use sempre!)
nome = 'Rodrigo'
idade = 30
print(f'Olá, {nome}! Você tem {idade} anos.')
print(f'Daqui 5 anos: {idade + 5}')
print(f'Preço: R${49.90:.2f}') # R$49.90
print(f'{nome:>20}') # Alinha à direita
```

MÉTODO	O QUE FAZ	EXEMPLO
<code>.upper()</code> / <code>.lower()</code>	Maiúsculas / minúsculas	'Olá'.upper() → 'OLÁ'
<code>.strip()</code>	Remove espaços das pontas	' oi '.strip() → 'oi'
<code>.split(sep)</code>	Divide em lista	'a,b,c'.split(',') → ['a','b','c']
<code>.join(lista)</code>	Junta lista em string	',' .join(['a','b']) → 'a, b'
<code>.replace(old, new)</code>	Substitui texto	'olá mundo'.replace('mundo','Python')
<code>.startswith()</code> / <code>.endswith()</code>	Verifica início/fim	'arquivo.pdf'.endswith('.pdf') → True
<code>.find(sub)</code>	Posição da substring (-1 se não achar)	'Python'.find('th') → 2
<code>.count(sub)</code>	Conta ocorrências	'banana'.count('a') → 3
<code>len(s)</code>	Tamanho da string	len('Python') → 6

Dica: Sempre use f-strings (f'...') para formatação. É o método mais rápido e legível. `.format()` e `%` são legados.

05 Estruturas Condicionais (if/elif/else)

Condicionais permitem que o programa tome decisões baseadas em condições. Em Python, usamos if, elif (else if) e else.

```
# Básico
idade = 18
if idade >= 18:
    print('Maior de idade')
elif idade >= 12:
    print('Adolescente')
else:
    print('Criança')

# Operador ternário
status = 'adulto' if idade >= 18 else 'menor'

# Match/case (Python 3.10+) - switch/case do Python
comando = 'sair'
match comando:
    case 'iniciar':
        print('Iniciando...')
    case 'pausar':
        print('Pausando...')
    case 'sair':
        print('Saindo...')
    case _:
        print('Comando desconhecido')
```

Valores falsy em Python:

VALOR	TIPO	AVALIA COMO
False	bool	Falso
0, 0.0	int, float	Falso
'' , "" , b''	str, bytes	Falso (string vazia)
[], (), {}, set()	coleções	Falso (coleção vazia)
None	NoneType	Falso
Qualquer outro valor	qualquer	Verdadeiro

Dica: Use 'if lista:' em vez de 'if len(lista) > 0:'. Python avalia coleções vazias como False automaticamente.

06 Estruturas de Repetição (for/while)

Loops permitem repetir blocos de código. Python oferece for (iteração) e while (repetição condicional).

```
# for - iterar sobre sequências
frutas = ['maçã', 'banana', 'uva']
for fruta in frutas:
    print(fruta)

# for com range
for i in range(5): # 0, 1, 2, 3, 4
    print(i)
for i in range(2, 10, 2): # 2, 4, 6, 8
    print(i)

# for com enumerate (índice + valor)
for i, fruta in enumerate(frutas):
    print(f'{i}: {fruta}')

# while
contador = 0
while contador < 5:
    print(contador)
    contador += 1

# break e continue
for n in range(100):
    if n == 5: break # para o loop
    if n % 2 == 0: continue # pula para o próximo
    print(n) # imprime: 1, 3
```

FUNÇÃO	O QUE FAZ	EXEMPLO
range(n)	Gera 0 até n-1	range(5) → 0,1,2,3,4
range(a, b)	Gera a até b-1	range(2,5) → 2,3,4
range(a, b, step)	Gera com passo	range(0,10,3) → 0,3,6,9
enumerate(seq)	Índice + valor	enumerate(['a','b']) → (0,'a'),(1,'b')
zip(a, b)	Itera em paralelo	zip([1,2],['a','b']) → (1,'a'),(2,'b')
reversed(seq)	Ordem inversa	reversed([1,2,3]) → 3,2,1

Dica: Prefira 'for item in lista' em vez de 'for i in range(len(lista))'. Se precisar do índice, use enumerate().

07 Funções e Escopo

Funções são blocos reutilizáveis de código. Em Python, funções são objetos de primeira classe - podem ser passadas como argumentos, retornadas por outras funções e atribuídas a variáveis.

```
# Função básica
def saudacao(nome: str) -> str:
    '''Retorna saudação personalizada.'''
    return f'Olá, {nome}!'

# Parâmetros com valor padrão
def criar_usuario(nome: str, idade: int = 0, ativo: bool = True):
    return {'nome': nome, 'idade': idade, 'ativo': ativo}

# *args e **kwargs
def soma(*numeros): # aceita qualquer qtd de args
    return sum(numeros)
soma(1, 2, 3, 4) # 10

def config(**opcoes): # aceita qualquer qtd de kwargs
    for chave, valor in opcoes.items():
        print(f'{chave} = {valor}')
config(tema='escuro', idioma='pt-BR')

# Função retornando múltiplos valores
def dividir(a, b):
    return a // b, a % b # retorna tupla
quociente, resto = dividir(10, 3) # 3, 1
```

CONCEITO	DESCRIÇÃO	EXEMPLO
def	Define função	def minha_funcao():
return	Retorna valor	return resultado
*args	Argumentos posicionais variáveis	def f(*args): ...
kwargs	Argumentos nomeados variáveis	def f(kwargs): ...
docstring	Documentação da função	'''Descrição da função.'''
scope local	Variáveis dentro da função	Existem só dentro da função
scope global	Variáveis no módulo	Acessíveis em todo o arquivo

Dica: Funções devem fazer UMA coisa e fazer bem. Se a função tem mais de 20 linhas, considere dividir em funções menores.

08 Listas, Tuplas e Conjuntos

Coleções são estruturas que armazenam múltiplos valores. Cada tipo tem suas vantagens.

Listas (list) - mutáveis, ordenadas:

```
frutas = ['maçã', 'banana', 'uva']
frutas.append('manga')    # Adiciona no final
frutas.insert(0, 'kiwi')  # Insere na posição
frutas.remove('banana')   # Remove por valor
frutas.pop()              # Remove o último
frutas.sort()             # Ordena
frutas.reverse()          # Inverte

# Slicing (fatiamento)
nums = [0, 1, 2, 3, 4, 5]
nums[1:4]    # [1, 2, 3]
nums[:3]     # [0, 1, 2]
nums[-2:]    # [4, 5]
nums[:2]     # [0, 1]
nums[::2]    # [0, 2, 4] (passo 2)
nums[::-1]   # [5, 4, 3, 2, 1, 0] (invertido)
```

TIPO	SINTAXE	MUTÁVEL	ORDENADA	DUPLICATAS	QUANDO USAR
list	[1, 2, 3]	Sim	Sim	Sim	Coleção geral, mais usada
tuple	(1, 2, 3)	Não	Sim	Sim	Dados imutáveis, retorno de função
set	{1, 2, 3}	Sim	Não	Não	Valores únicos, operações de conjunto
frozenset	frozenset({1,2})	Não	Não	Não	Set imutável (chave de dict)

Dica: Use tuplas para dados que não devem mudar (coordenadas, configurações). Use sets quando precisar de unicidade ou operações como união/interseção.

09 Dicionários

Dicionários são a estrutura de dados mais poderosa e versátil do Python. Armazenam pares chave:valor com acesso O(1).

```
# Criação
usuario = {'nome': 'Rodrigo', 'idade': 30, 'cidade': 'SP'}

# Acesso
print(usuario['nome'])           # 'Rodrigo'
print(usuario.get('email', 'N/A')) # 'N/A' (seguro)

# Modificação
usuario['email'] = 'rodrigo@email.com' # Adiciona/atualiza
del usuario['cidade']                 # Remove chave

# Iteração
for chave, valor in usuario.items():
    print(f'{chave}: {valor}')

# Merge (Python 3.9+)
defaults = {'tema': 'claro', 'idioma': 'en'}
custom = {'tema': 'escuro'}
config = defaults | custom # {'tema': 'escuro', 'idioma': 'en'}
```

MÉTODO	O QUE FAZ	EXEMPLO
<code>.get(key, default)</code>	Acessa com valor padrão	<code>d.get('x', 0) → 0</code> se não existir
<code>.keys()</code>	Retorna as chaves	<code>list(d.keys())</code>
<code>.values()</code>	Retorna os valores	<code>list(d.values())</code>
<code>.items()</code>	Retorna pares (chave, valor)	<code>for k, v in d.items():</code>
<code>.update(outro)</code>	Merge com outro dict	<code>d.update({'a': 1})</code>
<code>.pop(key)</code>	Remove e retorna valor	<code>d.pop('nome')</code>
<code>.setdefault(key, val)</code>	Retorna ou cria	<code>d.setdefault('lista', [])</code>

Dica: Use `.get()` em vez de `[]` para evitar `KeyError`. Use `|` para merge de dicts (Python 3.9+). Use `defaultdict` do `collections` para dicts com valor padrão.

10 Comprehensions

Comprehensions são uma forma concisa e 'pythônica' de criar listas, dicionários e conjuntos. São mais rápidas que loops equivalentes.

```
# List comprehension
quadrados = [x**2 for x in range(10)]
# [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

# Com filtro (if)
pares = [x for x in range(20) if x % 2 == 0]
# [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]

# Dict comprehension
nomes = ['Ana', 'Bruno', 'Carlos']
comprimentos = {nome: len(nome) for nome in nomes}
# {'Ana': 3, 'Bruno': 5, 'Carlos': 6}

# Set comprehension
unicos = {x % 5 for x in range(20)}
# {0, 1, 2, 3, 4}

# Generator expression (lazy, não cria lista na memória)
soma = sum(x**2 for x in range(1000000)) # eficiente!
```

TIPO	SINTAXE	RESULTADO
List comp	[expr for x in seq]	Lista
List comp + filtro	[expr for x in seq if cond]	Lista filtrada
Dict comp	{k: v for k, v in items}	Dicionário
Set comp	{expr for x in seq}	Conjunto
Generator	(expr for x in seq)	Gerador (lazy)

Dica: Use comprehensions para transformações simples (1 linha). Se a lógica ficar complexa (2+ condições, nested), prefira um loop for normal - legibilidade importa.

11 Manipulação de Arquivos

Python oferece formas simples e seguras de ler e escrever arquivos. O gerenciador de contexto 'with' garante que o arquivo será fechado corretamente.

```
# Escrever arquivo
with open('dados.txt', 'w', encoding='utf-8') as f:
    f.write('Linha 1\n')
    f.write('Linha 2\n')

# Ler arquivo inteiro
with open('dados.txt', 'r', encoding='utf-8') as f:
    conteudo = f.read()

# Ler linha por linha (eficiente para arquivos grandes)
with open('dados.txt', 'r') as f:
    for linha in f:
        print(linha.strip())

# JSON
import json
dados = {'nome': 'Rodrigo', 'skills': ['Python', 'IA']}
with open('dados.json', 'w') as f:
    json.dump(dados, f, indent=2, ensure_ascii=False)

# CSV
import csv
with open('dados.csv', 'w', newline='') as f:
    writer = csv.writer(f)
    writer.writerow(['Nome', 'Idade'])
    writer.writerow(['Rodrigo', 30])
```

MODO	DESCRIÇÃO
'r'	Leitura (padrão). Erro se o arquivo não existir
'w'	Escrita. Cria arquivo ou SOBRESCREVE o existente
'a'	Append. Adiciona ao final sem apagar
'x'	Criação exclusiva. Erro se já existir
'b'	Modo binário. Usar com 'rb', 'wb'
'r+'	Leitura e escrita

Dica: SEMPRE use 'with open(...)' em vez de f = open(...). O 'with' garante que o arquivo será fechado mesmo se ocorrer um erro. Sempre especifique encoding='utf-8'.

12 Tratamento de Erros (try/except)

Tratamento de exceções permite que o programa lide com erros de forma elegante em vez de simplesmente 'quebrar'.

```
# Básico
try:
    resultado = 10 / 0
except ZeroDivisionError:
    print('Divisão por zero!')

# Múltiplas exceções + else + finally
try:
    numero = int(input('Digite um número: '))
    resultado = 100 / numero
except ValueError:
    print('Isso não é um número!')
except ZeroDivisionError:
    print('Não pode dividir por zero!')
else:
    print(f'Resultado: {resultado}') # só executa se NÃO houve erro
finally:
    print('Sempre executa') # limpeza

# Levantar exceção personalizada
class SaldoInsuficienteError(Exception):
    pass

def sacar(saldo, valor):
    if valor > saldo:
        raise SaldoInsuficienteError(f'Saldo: {saldo}, Saque: {valor}')
    return saldo - valor
```

EXCEÇÃO	QUANDO OCORRE
<code>ValueError</code>	Tipo correto mas valor inválido: <code>int('abc')</code>
<code>TypeError</code>	Operação com tipo errado: <code>'a' + 1</code>
<code>KeyError</code>	Chave não existe no dicionário: <code>d['inexistente']</code>
<code>IndexError</code>	Índice fora do range: <code>lista[100]</code>
<code>FileNotFoundError</code>	Arquivo não encontrado: <code>open('nao_existe.txt')</code>
<code>AttributeError</code>	Atributo/método não existe: <code>None.upper()</code>
<code>ImportError</code>	Módulo não encontrado: <code>import inexistente</code>
<code>ZeroDivisionError</code>	Divisão por zero: <code>10 / 0</code>

Dica: Nunca use 'except:' sem especificar o tipo de exceção. Isso captura TUDO (até Ctrl+C) e esconde bugs.

13 Módulos e Pacotes (import, pip)

Módulos são arquivos .py que contêm código reutilizável. Pacotes são coleções de módulos. O PyPI (Python Package Index) tem 400.000+ pacotes disponíveis.

```
# Formas de import
import os                # módulo inteiro
from os import path      # item específico
from os.path import join, exists # múltiplos itens
import numpy as np       # alias (apelido)

# Instalar pacotes
# pip install requests pandas numpy
# uv add requests pandas numpy # moderno, mais rápido
```

Módulos da biblioteca padrão essenciais:

MÓDULO	FUNÇÃO	EXEMPLO
os / pathlib	Sistema de arquivos	Path('dados').mkdir(exist_ok=True)
json	Ler/escrever JSON	json.loads('{ "a": 1 }')
datetime	Datas e horários	datetime.now(), timedelta(days=7)
collections	Estruturas de dados extras	Counter, defaultdict, namedtuple
itertools	Iteradores avançados	chain, product, combinations
functools	Ferramentas para funções	lru_cache, partial, reduce
re	Expressões regulares	re.findall(r'\d+', texto)
logging	Sistema de logs	logging.info('Processando...')
typing	Anotações de tipo	List[str], Dict[str, int], Optional
dataclasses	Classes de dados	@dataclass para classes simples

Dica: Use pathlib em vez de os.path para manipulação de caminhos. É mais moderno e legível: Path('pasta') / 'arquivo.txt'.

14 Programação Orientada a Objetos (POO)

POO organiza código em classes e objetos. Classes são 'moldes' que definem atributos (dados) e métodos (comportamentos). Em Python, tudo é um objeto.

```
from dataclasses import dataclass

# Classe tradicional
class ContaBancaria:
    def __init__(self, titular: str, saldo: float = 0):
        self.titular = titular
        self.saldo = saldo

    def depositar(self, valor: float):
        self.saldo += valor
        return self

    def sacar(self, valor: float):
        if valor > self.saldo:
            raise ValueError('Saldo insuficiente')
        self.saldo -= valor
        return self

    def __str__(self):
        return f'{self.titular}: R${self.saldo:.2f}'

# Dataclass (moderno, menos boilerplate)
@dataclass
class Produto:
    nome: str
    preco: float
    estoque: int = 0

    @property
    def disponivel(self) -> bool:
        return self.estoque > 0
```

CONCEITO	DESCRIÇÃO	PYTHON
Classe	Molde para objetos	class MinhaClasse:
Objeto	Instância de uma classe	obj = MinhaClasse()
__init__	Construtor	def __init__(self, ...):
self	Referência ao objeto	self.atributo = valor
Herança	Classe filha herda da pai	class Filha(Pai):
@property	Atributo computado	Acessa como atributo, executa como método
@dataclass	Classe de dados automática	Gera __init__, __repr__, __eq__ etc.
Dunder methods	Métodos especiais __x__	__str__, __len__, __eq__, __lt__

Dica: Use @dataclass para classes que são principalmente 'containers de dados'. Para lógica complexa, use classes tradicionais. Prefira composição sobre herança.

15 Decoradores e Geradores

Decoradores modificam o comportamento de funções. Geradores produzem valores sob demanda (lazy evaluation), economizando memória.

Decoradores:

```
import time
from functools import wraps

# Decorador para medir tempo de execução
def timer(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        inicio = time.time()
        resultado = func(*args, **kwargs)
        fim = time.time()
        print(f'{func.__name__}: {fim - inicio:.3f}s')
        return resultado
    return wrapper

@timer
def processar_dados(n):
    return sum(range(n))

processar_dados(10_000_000)  # processar_dados: 0.234s
```

Geradores:

```
# Gerador com yield
def fibonacci(n):
    a, b = 0, 1
    for _ in range(n):
        yield a  # retorna e PAUSA
        a, b = b, a + b

# Uso - gera valores sob demanda
for num in fibonacci(10):
    print(num)  # 0, 1, 1, 2, 3, 5, 8, 13, 21, 34

# Gerador para arquivo grande (não carrega tudo na memória)
def ler_linhas(arquivo):
    with open(arquivo) as f:
        for linha in f:
            yield linha.strip()
```

Dica: Use `@functools.lru_cache` para cache automático de funções puras: resultados são guardados para evitar recalcul.

16 Lambda e Funções de Alta Ordem

Funções lambda são funções anônimas de uma linha. Funções de alta ordem recebem ou retornam outras funções.

```
# Lambda: função anônima
dobro = lambda x: x * 2
dobro(5) # 10

# map() - aplica função a cada item
nomes = ['ana', 'bruno', 'carlos']
maiusculas = list(map(str.upper, nomes))
# ['ANA', 'BRUNO', 'CARLOS']

# filter() - filtra por condição
numeros = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
pares = list(filter(lambda x: x % 2 == 0, numeros))
# [2, 4, 6, 8, 10]

# sorted() com key
alunos = [('Ana', 9.5), ('Bruno', 7.0), ('Carlos', 8.5)]
por_nota = sorted(alunos, key=lambda a: a[1], reverse=True)
# [('Ana', 9.5), ('Carlos', 8.5), ('Bruno', 7.0)]

# reduce() - acumula resultado
from functools import reduce
produto = reduce(lambda a, b: a * b, [1, 2, 3, 4, 5]) # 120
```

Dica: Prefira list comprehension sobre map/filter na maioria dos casos: `[x*2 for x in nums]` é mais 'pythônico' que `list(map(lambda x: x*2, nums))`.

17 Type Hints e Tipagem Estática

Type hints (anotações de tipo) tornam o código mais legível e permitem detectar erros antes da execução com ferramentas como mypy e Pyright.

```
# Tipos básicos
nome: str = 'Rodrigo'
idade: int = 30
ativo: bool = True

# Funções tipadas
def saudar(nome: str, vezes: int = 1) -> str:
    return f'Olá, {nome}! ' * vezes

# Tipos complexos (Python 3.10+)
def processar(dados: list[dict[str, int]]) -> dict[str, float]:
    ...

# Union e Optional
def buscar(id: int) -> str | None: # Python 3.10+
    ...

# TypedDict para dicts estruturados
from typing import TypedDict

class Usuario(TypedDict):
    nome: str
    idade: int
    email: str | None
```

TIPO	SINTAXE MODERNA (3.10+)	EXEMPLO
Lista tipada	list[str]	nomes: list[str] = ['Ana', 'Bruno']
Dict tipado	dict[str, int]	idades: dict[str, int] = {'Ana': 25}
Opcional	str None	email: str None = None
Union	int float	valor: int float = 3.14
Tupla	tuple[str, int]	par: tuple[str, int] = ('Ana', 25)
Callable	Callable[[int], str]	Função que recebe int e retorna str

Dica: Type hints NÃO afetam a execução (são ignorados em runtime). Seu valor está na documentação do código e na verificação com mypy/Pyright.

18 Ambientes Virtuais e Dependências

Ambientes virtuais isolam as dependências de cada projeto. Isso evita conflitos entre versões de pacotes.

FERRAMENTA	COMANDO	VANTAGEM
venv (padrão)	python -m venv .venv	Vem com Python, simples
uv (moderno)	uv init / uv add pandas	10-100x mais rápido, resolver avançado
poetry	poetry init / poetry add pandas	Gerencia projeto completo, lock file
conda	conda create -n meuenv	Ciência de dados, binários complexos
pipenv	pipenv install pandas	Pipfile + lock, virtual env automático

Workflow com uv (recomendado em 2026):

```
# Criar projeto
uv init meu-projeto
cd meu-projeto

# Adicionar dependências
uv add fastapi uvicorn
uv add --dev pytest ruff mypy

# Rodar script
uv run python main.py

# Rodar pytest
uv run pytest
```

Workflow com venv (padrão):

```
# Criar ambiente virtual
python -m venv .venv

# Ativar
# Linux/macOS: source .venv/bin/activate
# Windows: .venv\Scripts\activate

# Instalar pacotes
pip install fastapi uvicorn

# Salvar dependências
pip freeze > requirements.txt

# Instalar de requirements.txt
pip install -r requirements.txt
```

Dica: NUNCA instale pacotes globalmente (sem venv). Cada projeto deve ter seu ambiente isolado. Em 2026, uv é a ferramenta recomendada.

19 Programação Assíncrona (async/await)

Programação assíncrona permite executar múltiplas operações de I/O (rede, disco) concorrentemente sem threads. É essencial para web servers, crawlers e APIs de alta performance.

```
import asyncio
import httpx # cliente HTTP async

# Função assíncrona
async def buscar_dados(url: str) -> dict:
    async with httpx.AsyncClient() as client:
        response = await client.get(url)
        return response.json()

# Executar múltiplas chamadas em paralelo
async def main():
    urls = [
        'https://api.example.com/users',
        'https://api.example.com/products',
        'https://api.example.com/orders',
    ]
    # asyncio.gather executa todas ao mesmo tempo
    resultados = await asyncio.gather(
        *[buscar_dados(url) for url in urls]
    )
    return resultados

# Rodar
asyncio.run(main())
```

CONCEITO	DESCRIÇÃO
async def	Define uma função assíncrona (coroutine)
await	Espera o resultado de uma coroutine
asyncio.run()	Ponto de entrada para código async
asyncio.gather()	Executa múltiplas coroutines em paralelo
asyncio.create_task()	Agenda coroutine para execução concorrente
async with	Gerenciador de contexto assíncrono
async for	Iteração assíncrona

Dica: Async não é 'mais rápido' que sync para tudo. É ótimo para I/O (rede, disco). Para CPU (cálculos), use multiprocessing ou threading.

20 Testes Automatizados (pytest)

Testes automatizados garantem que o código funciona corretamente e continua funcionando após mudanças. pytest é o framework padrão do ecossistema Python.

```
# calculadora.py
def somar(a: float, b: float) -> float:
    return a + b

def dividir(a: float, b: float) -> float:
    if b == 0:
        raise ValueError('Divisão por zero')
    return a / b

# test_calculadora.py
import pytest
from calculadora import somar, dividir

def test_somar():
    assert somar(2, 3) == 5
    assert somar(-1, 1) == 0
    assert somar(0, 0) == 0

def test_dividir():
    assert dividir(10, 2) == 5.0

def test_dividir_por_zero():
    with pytest.raises(ValueError):
        dividir(10, 0)

# Rodar: pytest -v
```

RECURSO PYTEST	DESCRIÇÃO	COMANDO
Rodar todos	Executa todos os testes	pytest
Verbose	Detalhes de cada teste	pytest -v
Específico	Um arquivo ou função	pytest test_calc.py::test_somar
Coverage	Cobertura de código	pytest --cov=src
Fixtures	Setup/teardown reutilizável	@pytest.fixture
Parametrize	Múltiplos inputs	@pytest.mark.parametrize

Dica: Escreva testes ANTES ou JUNTO com o código, não depois. Objetivo mínimo: 80% de cobertura nos caminhos críticos.

21 Novidades do Python 3.13 e 3.14

Python 3.13 (outubro 2024) e 3.14 (outubro 2025) trouxeram mudanças significativas que melhoram performance, usabilidade e capacidades da linguagem.

Python 3.13:

NOVIDADE	DESCRIÇÃO	IMPACTO
REPL melhorado	Terminal interativo baseado no PyPy com edição multilinhas e cores	Melhor experiência de aprendizado e debug
Tracebacks coloridos	Erros com cores e indicação precisa do problema	Debug mais rápido e intuitivo
Free-threaded (experimental)	Modo sem GIL para paralelismo real	Performance em workloads CPU-bound
JIT experimental	Compilador JIT para otimização de runtime	Performance para código numérico
Deprecações	Removidos módulos legados (cgi, mailbox)	Código mais limpo

Python 3.14:

NOVIDADE	DESCRIÇÃO	IMPACTO
Template Strings (t-strings)	Novo tipo de string literal para templates seguros	Previne SQL injection, XSS
Deferred Annotations (PEP 649)	Anotações de tipo avaliadas sob demanda	Menos erros com forward references
Subinterpreters stdlib	Suporte nativo a subinterpreters	Paralelismo real sem GIL
compression.zstd	Compressão Zstandard na stdlib	Compressão rápida sem instalar pacotes
Free-threaded melhorado	GIL opcional mais estável, PEP 703 completo	Multi-threading performante
REPL com syntax highlight	Cores e highlighting no terminal interativo	Experiência de desenvolvimento melhor

Dica: A maior mudança de longo prazo é o free-threaded mode (sem GIL). Isso muda fundamentalmente como Python lida com concorrência e pode dobrar a performance de apps multi-threaded.

22 Ciência de Dados

Python é a linguagem padrão para ciência de dados. O ecossistema inclui ferramentas para manipulação, análise, visualização e machine learning.

BIBLIOTECA	FUNÇÃO	USO PRINCIPAL
NumPy	Arrays numéricos, álgebra linear	Base de cálculo numérico
Pandas	DataFrames, manipulação de dados tabulares	Limpeza, transformação, análise
Matplotlib	Gráficos e visualizações	Gráficos 2D, publicação científica
Seaborn	Visualização estatística	Gráficos bonitos com menos código
Plotly	Gráficos interativos	Dashboards, apresentações
scikit-learn	Machine Learning	Classificação, regressão, clustering
Polars	DataFrames ultra-rápidos	Alternativa moderna ao Pandas

Exemplo rápido com Pandas:

```
import pandas as pd

# Ler CSV
df = pd.read_csv('vendas.csv')

# Explorar
df.head()      # primeiras 5 linhas
df.describe()  # estatísticas básicas
df.info()      # tipos e nulos

# Filtrar e agregar
vendas_sp = df[df['estado'] == 'SP']
por_mes = df.groupby('mes')['valor'].sum()

# Criar coluna calculada
df['lucro'] = df['receita'] - df['custo']
```

Dica: Em 2026, considere Polars como alternativa ao Pandas para datasets grandes. É 10-50x mais rápido e usa menos memória.

23 Desenvolvimento Web

Python possui frameworks web para todos os casos: desde APIs simples até aplicações enterprise completas.

FRAMEWORK	TIPO	QUANDO USAR
FastAPI	API moderna (async)	APIs REST/GraphQL, microsserviços, IA. Mais popular em 2026
Django	Full-stack (batteries-included)	Apps completas com admin, auth, ORM. Enterprise
Flask	Micro-framework	APIs simples, prototipagem rápida
Litestar	API async (alternativa ao FastAPI)	APIs de alta performance
Streamlit	Apps de dados	Dashboards, protótipos de ML, demos

FastAPI - Hello World:

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    nome: str
    preco: float
    em_estoque: bool = True

@app.get('/')
def root():
    return {'msg': 'API funcionando!'}

@app.post('/items/')
def criar_item(item: Item):
    return {'item': item, 'status': 'criado'}
```

```
# Rodar: uvicorn main:app --reload
# Docs automática: http://localhost:8000/docs
```

Dica: FastAPI gera documentação OpenAPI automaticamente. Acesse /docs para Swagger UI e /redoc para ReDoc. Use Pydantic para validação de dados.

24 Automação e Web Scraping

Python é a linguagem mais usada para automação de tarefas e extração de dados da web.

BIBLIOTECA	FUNÇÃO	QUANDO USAR
requests	Chamadas HTTP	APIs, downloads, requisições simples
httpx	HTTP async + sync	Alternativa moderna ao requests
BeautifulSoup	Parsing de HTML	Extrair dados de páginas web
Selenium	Automação de navegador	Sites com JavaScript, login, interação
Playwright	Automação moderna de browser	Alternativa ao Selenium, mais rápido
Scrapy	Framework de crawling	Scraping em larga escala
schedule	Agendamento de tarefas	Executar scripts periodicamente
openpyxl	Manipular Excel	Automatizar planilhas

Exemplo - Web Scraping com requests + BeautifulSoup:

```
import requests
from bs4 import BeautifulSoup

# Buscar página
url = 'https://example.com/produtos'
response = requests.get(url)
soup = BeautifulSoup(response.text, 'html.parser')

# Extrair dados
produtos = []
for card in soup.select('.product-card'):
    nome = card.select_one('.name').text.strip()
    preco = card.select_one('.price').text.strip()
    produtos.append({'nome': nome, 'preco': preco})

print(f'Encontrados {len(produtos)} produtos')
```

Dica: Sempre verifique os termos de uso do site antes de fazer scraping. Use delays entre requisições e respeite o robots.txt.

25 IA e Machine Learning

Python é a linguagem dominante em inteligência artificial. Desde ML clássico até LLMs, o ecossistema Python cobre tudo.

ÁREA	BIBLIOTECAS	EXEMPLOS DE USO
ML Clássico	scikit-learn, XGBoost, LightGBM	Classificação, regressão, clustering
Deep Learning	PyTorch, TensorFlow, JAX	Redes neurais, visão computacional
NLP / LLMs	LangChain, Hugging Face, spaCy	Chatbots, RAG, análise de texto
Visão Computacional	OpenCV, Pillow, ultralytics	Detecção de objetos, OCR
Dados Tabulares	Pandas, Polars, DuckDB	ETL, análise, feature engineering
MLOps	MLflow, Weights & Biases, DVC	Tracking, versionamento, deploy

Exemplo - ML com scikit-learn:

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
import pandas as pd

# Carregar dados
df = pd.read_csv('clientes.csv')
X = df.drop('churn', axis=1) # features
y = df['churn']             # target

# Dividir treino/teste
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Treinar modelo
model = RandomForestClassifier(n_estimators=100)
model.fit(X_train, y_train)

# Avaliar
pred = model.predict(X_test)
print(f'Acurácia: {accuracy_score(y_test, pred):.2%}')
```

Dica: Comece com scikit-learn para ML clássico. Só vá para PyTorch/TensorFlow quando precisar de deep learning. Para LLMs, use LangChain.

26 APIs REST com FastAPI

FastAPI é o framework mais popular para APIs em Python em 2026. Combina alta performance (async), validação automática (Pydantic) e documentação autogerada.

CRUD completo:

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI(title='Minha API', version='1.0')

class TaskCreate(BaseModel):
    titulo: str
    descricao: str | None = None

class Task(TaskCreate):
    id: int
    feita: bool = False

tasks: dict[int, Task] = {}
next_id = 1

@app.get('/tasks', response_model=list[Task])
def listar(): return list(tasks.values())

@app.post('/tasks', response_model=Task, status_code=201)
def criar(data: TaskCreate):
    global next_id
    task = Task(id=next_id, **data.model_dump())
    tasks[next_id] = task
    next_id += 1
    return task

@app.put('/tasks/{id}')
def atualizar(id: int, data: TaskCreate):
    if id not in tasks: raise HTTPException(404)
    tasks[id] = Task(id=id, **data.model_dump())
    return tasks[id]

@app.delete('/tasks/{id}', status_code=204)
def deletar(id: int):
    if id not in tasks: raise HTTPException(404)
    del tasks[id]
```

Dica: Use SQLAlchemy + Alembic para banco de dados real, e adicione autenticação com OAuth2/JWT para produção.

27 Boas Práticas e Produtividade

Dominar boas práticas é o que separa um iniciante de um profissional. Estas dicas valem para qualquer projeto Python.

PRÁTICA	DETALHES
Use <code>type hints</code>	Documente tipos em funções. Ajuda você, seu editor e outros devs
Use <code>f-strings</code>	<code>f'{variavel}'</code> é o método mais rápido e legível de formatação
Use <code>pathlib</code>	<code>Path('pasta') / 'arquivo'</code> em vez de <code>os.path.join()</code>
Use <code>dataclasses</code>	<code>@dataclass</code> para classes simples. Menos boilerplate
Use <code>comprehensions</code>	Mais conciso e rápido que <code>for</code> + <code>append</code>
Use <code>with</code> (<code>context manager</code>)	Garante limpeza de recursos (arquivos, conexões)
Use <code>uv</code>	Gerenciador moderno, rápido, substitui <code>pip+venv+pip-tools</code>
Use <code>Ruff</code>	Lintor + formatter em um só. Substitui <code>flake8+black+isort</code>
Escreva testes	<code>pytest</code> para testes. Mínimo 80% de cobertura em código crítico
Use <code>logging</code>	<code>logging.info()</code> em vez de <code>print()</code> para produção

Estrutura de projeto recomendada:

```
meu-projeto/  
  pyproject.toml    # Config do projeto (uv/poetry)  
  README.md  
  src/  
    meu_projeto/  
      __init__.py  
      main.py      # Ponto de entrada  
      models.py    # Modelos de dados  
      services.py  # Lógica de negócio  
      utils.py     # Funções utilitárias  
  tests/  
    test_main.py  
    test_services.py  
  .env              # Variáveis de ambiente (não commite!)  
  .gitignore
```

O Zen of Python (import this):

PRINCÍPIO	SIGNIFICADO PRÁTICO
Beautiful is better than ugly	Escreva código limpo e bem formatado
Explicit is better than implicit	Seja claro nas intenções do código
Simple is better than complex	Não complique o que pode ser simples
Readability counts	Código é lido mais vezes do que escrito
Errors should never pass silently	Trate exceções, não as ignore
There should be one obvious way	Siga os padrões da comunidade

Dica final: A melhor forma de aprender Python é praticando. Escolha um projeto pessoal, construa, erre, aprenda. Participe da comunidade Python Brasil!

Conclusão

Este guia cobriu Python desde os fundamentos (variáveis, loops, funções) até aplicações avançadas (ciência de dados, web, IA, automação). Python é mais do que uma linguagem - é um ecossistema completo que permite construir praticamente qualquer coisa. Com as boas práticas e ferramentas apresentadas aqui, você está preparado para criar projetos profissionais e produtivos.

Guia gerado em Março 2026 por Claude para Rodrigo. Versão no tema escuro da Formação Claude Code 2026: conteúdo original mantido, com acentuação e recuo do código restaurados; os exemplos que usam só a biblioteca padrão foram executados com sucesso.